Agentic Workflow — Complete Reference

Last updated: 2026-08-16 · Living document, edit this directly for day-to-day changes. As of 2026-08-16 this reference is version-controlled at `docs/agentic-workflow.md` inside the `dotfiles` repo (it previously sat loose in the untracked parent folder), so it clones to a fresh machine with the rest of the setup. `agentic-workflow.pdf` is a periodic polished snapshot generated from this file's content; only worth regenerating after a batch of real changes, not every edit. Regenerate it with `docs/build-pdf.sh` (Markdown -> HTML -> headless-Chrome print-to-PDF).

> **Current state:** fully built and verified working. WezTerm + herdr + Neovim + Claude Code + Codex + OpenCode, all wired to local (`omlx`) and cloud (Claude) inference, voice input via Parakeet v3, the full axi tool family, and Kun's productivity stack (lavish, treehouse, no-mistakes, gnhf, firstmate) are all installed, declared in Nix, and tested end-to-end. GitHub Copilot CLI (`gc`) was added 2026-07-11 and authenticated 2026-08-16, now verified working end-to-end, see §2. Codex's default local model swapped again 2026-07-12, to `mlx-community--Qwen3-Coder-Next-8bit` — see §5. That session also uncovered and fixed a real bug in `rebuild.sh` (missing `--impure`) that had been silently zeroing out `OMLX_API_KEY` on every rebuild — see §9.

---

1. Quick Reference — Keys & Commands

The one mental model: a prefix key (herdr = `Ctrl+b`) precedes multiplexer commands. In Neovim, a leader key (`Space`) precedes custom commands. Press prefix/leader, release, then the next key.

herdr — prefix Ctrl+b

Key	Action
`Ctrl+b "`	split pane horizontal
`Ctrl+b %`	split pane vertical
`Ctrl+b h/j/k/l`	move focus
`Ctrl+b c`	new tab (fresh shell)
`Ctrl+b &`	close tab
`Ctrl+b w`	workspace picker
`Ctrl+b g`	go to (jump)
`Ctrl+b y`	copy mode (v/Space select, y/Enter copy)

Sessions persist on a background herdr server — closing the terminal doesn't kill your work. But see §9 for why that same persistence can cause stale-environment bugs.

Neovim — leader Space

Key	Action
`h j k l`	left/down/up/right
`w / b`	next/prev word
`0 / $`	start/end of line
`gg / G`	top/bottom of file
`Nj / Nk`	jump N lines (relative #s in gutter)
`i / a / o / O`	insert modes
`Esc`	leave insert + saves (custom map)
`dd / yy / p`	delete / copy / paste line
`u / Ctrl+r`	undo / redo

Key	Action
`Space f`	find files
`Space s`	search/grep project
`Space b`	switch buffers
`Space e`	file browser (Oil)
`Space g`	git UI (Neogit)
`Space m`	markdown preview, floating window (glow.nvim), `:q` to close
`gd`	go to definition

Agents

Alias	Command
`cc`	`claude --dangerously-skip-permissions` (cloud)
`co`	`codex --sandbox workspace-write --ask-for-approval never` (routed to local `omlx`, default model `mlx-community--Qwen3-Coder-Next-8bit` as of 2026-07-12 — see §5)
`oc`	`omlx launch opencode --model mlx-community--Qwen3.6-35B-A3B-8bit` (routed to local `omlx`). Still on the older default — not yet swapped to Qwen3-Coder-Next, see §5. Prefer this over `co` for context-heavy work regardless.
`gc`	`copilot --allow-all` (cloud, GitHub Copilot subscription). Added 2026-07-11, authenticated 2026-08-16 and working, see §2.

`/` — slash commands (skills, config, `/lavish`, `/no-mistakes`). Plan mode — ask the agent to plan before building. `Esc` — interrupt agent mid-task.

Voice input (Parakeet v3)

- hold `⌥ Option Right` — push-to-talk: hold, speak, release
- `⌥ Option + `` — toggle: press to start, press to stop

Global hotkey — works in any focused app (herdr pane, Messages, anywhere), agent-agnostic. No vocab-priming with Parakeet (Whisper-only feature) — see §6.

Git essentials

Command	Action
`git status`	what changed
`add`	alias: `git add .`
`git commit -m`	commit staged
`push / pull`	aliases for `git push/pull`
`m`	alias: `git switch main`
`lazygit`	full-screen git TUI

Protocol: HTTPS via `gh auth login`, not SSH — background daemons (no-mistakes, gnhf, firstmate crew) can't reliably reach an SSH agent socket, but the gh-issued token in Keychain works for any process.

omlx (local inference)

Command	Action
`omlx status`	server/client health
`omlx restart`	restart + rescan for new models

Command	Action
`omlx launch <tool> --model <id>`	wire a tool to omlx
`hf download <repo>`	pull a new model (omlx's bundled CLI, `/opt/homebrew/Cellar/omlx/<version>/libexec/bin/hf`)

---

2. Component Status

Component	Status	Notes
WezTerm, herdr, Neovim, Claude Code, node/npx	done	core ship, unchanged since original setup
lavish	done	project + global skill; visual planning
skill-creator	done	Anthropic's own — lets agent author new skills
gh-axi	done	GitHub ops, ~3x cheaper/2x faster than GitHub MCP
chrome-devtools-axi	done	browser automation, ~40% fewer tokens than raw MCP
quota-axi	done	usage tracking across Claude/Codex/Cursor/Copilot/Grok
treehouse	done	pooled git worktrees, ~/.local/bin/treehouse
no-mistakes	done	validation pipeline, daemon running
gnhf	done	"good night, have fun" — the unattended overnight autonomous loop. Via `npx gnhf` (no global install — see §9)
firstmate	done	cloned to `~/github/kunchenguid/firstmate`; fresh-machine re-clone step now documented in the dotfiles README
gh (GitHub CLI)	done	authenticated, HTTPS protocol
omlx	done	v0.5.0. GPU memory ceiling + context-window policy hardened 2026-07-10 — see §5
Codex CLI → omlx	done	wired to omlx, verified via `codex exec`. Bundled model catalog doesn't recognize local models — cosmetic metadata warning only, see §5
OpenCode (oc) → omlx	done	wired via `omlx launch opencode`; preferred over Codex for context-heavy sessions since omlx writes real metadata into its config — see §5
Claude Code → omlx	decided	deliberately cloud-only — not wired, by choice
GitHub Copilot CLI (`gc`)	done	cask `copilot-cli` declared in `configuration.nix`, alias in `home.nix`, installed via `./rebuild.sh` 2026-07-11. Authenticated via `copilot login` and verified working end-to-end 2026-08-16. Cloud-only for now; it does support BYOK to an OpenAI-compatible endpoint via `COPILOT_PROVIDER_BASE_URL` (`copilot help providers`), so wiring it to `omlx` like `co` / `oc` is possible but not yet attempted — see §5. No global instructions file (no `~/.copilot/AGENTS.md` equivalent) — see §3.
Markdown preview (`glow` + `glow.nvim`, `Space m`)	pending	`glow` added to `home.packages` in `home.nix` 2026-07-11; `glow.nvim` added as new `lua/plugins/markdown.lua`, lazy-loaded on first `:Glow` call. Declared, not yet applied - run `./rebuild.sh` to install `glow`, then launch `nvim` once so lazy.nvim fetches the plugin.
leetcode.nvim (`:Leet`)	done	`kawre/leetcode.nvim` added as `lua/plugins/leetcode.lua` 2026-07-11 — no Nix/`rebuild.sh` involved, this repo's Neovim plugins are managed entirely by `lazy.nvim` (installs from GitHub on next `nvim` launch). `lang = 'python3'`, `plugins.non_standalone = true` so `:Leet` works inside a normal session with other buffers open — close it with `:Leet exit`, not `:q`. Sign-in needs the full `Cookie` request header (`csrftoken` + `LEETCODE_SESSION`) copied from DevTools Network tab, not `Set-Cookie` — see §9 gotchas for the paste/Brave quirks.
OpenSuperWhisper + Parakeet v3	done	installed, permissions granted, verified pasting text

Component	Status	Notes
Kokoro TTS (voice output)	not built	bundled in omlx, unused — see §8

---

3. Memory Files & Skills — How the Agent Learns Your Preferences

Global memory (`~/.claude/CLAUDE.md`, symlinked to AGENTS.md for Codex/OpenCode too) — personal preferences, loaded into every session. Keep it short (~27 lines); it costs tokens on every request. Yours already has: never use em dash, don't weight development cost in technical decisions, reproduce bugs end-to-end before fixing, be pixel-perfect on UI, fix lint/test issues you notice even if unrelated to the task.

> **Copilot CLI is the odd one out:** as of 2026-07-11, GitHub's docs describe no user-level/global instructions file for `gc` (no `~/.copilot/AGENTS.md` equivalent to symlink `home/AGENTS.md` to). It does read per-repo `.github/copilot-instructions.md`, `.github/instructions/**/*.instructions.md`, and a repo-root `AGENTS.md` (this dotfiles repo already has one) — so it inherits project conventions, just not your personal global preferences. `copilot init` can scaffold the per-repo file. Revisit if GitHub adds a global option later.

Project memory (`CLAUDE.md` / `AGENTS.md` in a repo root) — grows by correcting the agent and telling it to "remember." This is the collective learning of every session in that project.

Skills — conditionally-loaded procedures (only read when relevant), installed via `npx skills add <owner/repo> --skill <name> [-g]`. Move verbose/conditional instructions out of memory files and into skills to keep the memory file lean.

> **Skill security:** a skill can run anything on your machine — API key exfiltration, arbitrary shell access. Do not install skills from unverified sources, even popular ones (a benchmark cited in Kun's video showed a 177k-star skill making agent output measurably worse while using 5% more tokens). Only the official kunchenguid axi catalog (axi.md) and Anthropic's own skills were installed here. Scanner "Med Risk" ratings on gh-axi / chrome-devtools-axi / quota-axi reflect that they can act on GitHub/a browser/your usage data — that's the capability working as intended, not a red flag on the code itself.

---

4. The Workflow Loop

1. **Plan** — Discuss in lavish (interactive HTML artifact, browser-based annotation) — decide requirements here, not by reading walls of text
2. **Build** — Agent works the middle — step away, start other tasks
3. **Validate** — no-mistakes: branch → adversarial review → e2e test (with evidence) → docs → lint → PR, babysat until merged
4. **Judge** — Review the PR, weighted by its risk assessment — not every diff
5. **Scale** — treehouse (parallel worktrees, no manual cleanup) + firstmate (manages the crew so you talk to one agent, not many)

> **Captain's mindset:** you're an engineering director, not a diff-reviewer. Set good process, trust the pipeline, spend your energy on what to build — competition, users, direction — not line-by-line review.

Daily muscle-memory drills: split a herdr pane, run an agent in one, nvim in another · jump with relative line numbers (`Nj` / `Nk`) instead of scrolling · `Space f` a file, `Space s` a symbol — no mouse, ever · dictate instead of type whenever the prompt isn't a URL/path · detach herdr, reattach — the session is still there.

Goal: hands never leave the keyboard. That's the whole point — staying in flow.

---

5. Local Inference — omlx

omlx is an MLX inference server purpose-built for coding agents on Apple Silicon: continuous batching, paged SSD KV-caching (repeated large contexts go from 30–90s time-to-first-token down to 1–3s), and native OpenAI + Anthropic API compatibility. Verified legitimate via a real discussion thread in Apple's own ml-explore/mlx GitHub repo.

Current setup

- Installed via private Homebrew tap `jundot/omlx`, trusted and declared in `configuration.nix`
- Running as a background service (`homebrew.mxcl.omlx` launchd job), auto-starts at login
- **Codex's default model as of 2026-07-12:** `mlx-community--Qwen3-Coder-Next-8bit` (80B total / 3B active MoE, 256K native context). Replaced `mlx-community--Qwen3.6-35B-A3B-8bit` — see "Model swap: → Qwen3-Coder-Next-8bit" below for rationale.
- `oc` /OpenCode is still on `mlx-community--Qwen3.6-35B-A3B-8bit` — not yet swapped, since that alias hardcodes its model in `home.nix` rather than reading `~/.codex/config.toml`. Revisit if Qwen3-Coder-Next proves out well through Codex.
- Codex CLI is wired to it (`~/.codex/config.toml`, provider `omlx`, base_url `http://127.0.0.1:8000/v1`) — verified working end-to-end via `codex exec`
- OpenCode is also wired to it via `omlx launch opencode --model <id>` (the `oc` alias) — preferred over Codex for context-heavy work, since omlx writes its real, live context-window metadata into OpenCode's config at launch time instead of Codex guessing from a static bundled catalog
- Claude Code stays cloud-only — a deliberate choice, not a gap.

> **Trust lesson from this setup:** a model already on disk (`...Claude-4.6-Opus-Deckard-Heretic-Uncensored-Thinking-8bit`) was deleted before use. "Heretic" is a known automated abliteration technique that strips safety/refusal training — it measurably degrades coherence and instruction-following even when done carefully, and there's no legitimate coding/planning task that benefits from an uncensored model. The name also falsely implied lineage with Claude Opus. Before running any downloaded model: check the name for uncensored/abliterated branding and fabricated lineage claims to closed frontier models. Prefer official mlx-community releases.

Model swap: Qwen3.6-27B-8bit → Qwen3.6-35B-A3B-8bit (2026-07-11)

Motivation: the dense 27B model was clearing ~16.8 tok/s through Codex, well under the ~30 tok/s target for a smooth interactive loop.

What was tried and ruled out:

- `antirez/ds4` ("DwarfStar") — a dedicated inference engine for DeepSeek V4 Flash (284B total / 13B active). Legitimate, MIT-licensed, real published benchmark of 34.27 tok/s / 25.9 tok/s (short/11.7K-context) on an M5 Max 128GB — the exact machine class here. Not adopted yet: beta-quality software, a separate engine/server from omlx (not a swap-in), and `ds4-agent` is explicitly alpha. Worth revisiting if the omlx-native option below stops being enough.
- GLM-5.2 and MiniMax-M3 top the open-weight intelligence leaderboards (Artificial Analysis) but are physically too large for 128GB (753B and 427B total params respectively, even at aggressive 4-bit quant).

What was adopted: `mlx-community/Qwen3.6-35B-A3B-8bit` — same family omlx already has custom kernels for (Qwen3.5/3.6), MoE with only ~3B active params per token, zero new integration (just `hf download` + `omlx restart`, no new alias structure, Codex/OpenCode stay wired exactly as before).

Measured results (own hardware, this M5 Max 128GB):

- Raw `mlx_lm.benchmark`, short prompt: **~104 tok/s** generation (vs ~16.8 tok/s baseline)
- Raw `mlx_lm.benchmark`, ~11.7K context: **~94 tok/s** generation — MoE barely degrades under long context, unlike the dense model
- Through the live omlx server (`/v1/chat/completions`): ~88 tok/s
- Real `codex exec` run, identical prompt on both models: 42.3s wall-clock (27B, prompt-cache warm) vs 14.3s (35B-A3B, prompt-cache cold — a harder case for it) while producing 53% *more* output tokens
- **Net: ~5.6x faster generation throughput, ~2.9x faster wall-clock on a real Codex task**

Quality tradeoff (Qwen's own published benchmarks, direct comparison of these two exact checkpoints):

Benchmark	Qwen3.6-27B (old default)	Qwen3.6-35B-A3B (new default)	Gap
SWE-bench Verified	77.2	73.4	-3.8
SWE-bench Pro	53.5	49.5	-4.0
Terminal-Bench 2.0	59.3	51.5	-7.8
LiveCodeBench v6	83.9	80.4	-3.5
MMLU-Pro	86.2	85.2	-1.0
GPQA Diamond	87.8	86.0	-1.8

Benchmark	Qwen3.6-27B (old default)	Qwen3.6-35B-A3B (new default)	Gap
AIME26	94.1	92.7	-1.4

The dense model wins every benchmark, but mostly by 1-4 points; Terminal-Bench (closest proxy to real agentic CLI tool-use) shows the largest gap at ~8 points. Judged worth it for the ~3-5.6x speed gain in an interactive loop where iteration speed compounds.

Fallback: the old 27B weights are kept on disk (not deleted). For the rare task where you want max quality over speed:

```
omlx launch codex --model mlx-community--Qwen3.6-27B-8bit
```

Model swap: Qwen3.6-35B-A3B-8bit → Qwen3-Coder-Next-8bit (2026-07-12, Codex only)

Motivation: newer Qwen3-Next hybrid architecture, still only ~3B active params (so generation speed stays close to the 35B-A3B default) but 80B total params and 256K native context vs the old model's much smaller window — worth it given 128GB unified memory has plenty of headroom for the 8bit quant (~79GB on disk).

Adopted: `mlx-community/Qwen3-Coder-Next-8bit` via omlx's bundled `hf` CLI (`/opt/homebrew/Cellar/omlx/<version>/libexec/bin/hf download mlx-community/Qwen3-Coder-Next-8bit`), same quant precision (8bit) as both prior defaults for consistency. `~/.codex/config.toml` updated: `model = "mlx-community--Qwen3-Coder-Next-8bit"` (provider stays `omlx`).

Verified: started the omlx server, confirmed the model in `GET /v1/models`, sent a direct `/v1/chat/completions` test request (loaded + responded correctly, ~12.4s cold load), then ran a real `codex exec` end-to-end (quicksort/mergesort/heapsort request) and got correct, well-formed output.

Measured results (own hardware, this M5 Max 128GB), same methodology as the earlier swap:

	Qwen3.6-27B-8bit (dense)	Qwen3.6-35B-A3B-8bit (MoE)	Qwen3-Coder-Next-8bit (MoE)
Short-context generation	~16.8 tok/s	~104 tok/s raw / ~88 tok/s server	~79 tok/s raw (`mlx_lm.benchmark -p 64 -g 256`)
~11.7K-context generation	not measured	~94 tok/s	~74 tok/s raw (`mlx_lm.benchmark -p 11700 -g 256`)
Peak memory	27GB-class	~37-39GB	~85-87GB
Real `codex exec` wall-clock (identical quicksort/mergesort/heapsort prompt)	42.3s	14.3s (cold)	9.0s (clean re-run — see caveat below)

Takeaways: Coder-Next is slower per-token than 35B-A3B raw (~74-79 tok/s vs ~88-104 tok/s) — expected, it's a much bigger model (80B vs 35B total) even at the same ~3B active params, so routing/attention overhead is higher. It still shares 35B-A3B's key property: barely slows down under long context (79→74 tok/s, ~6% drop) vs. how much a dense model degrades. The 9.0s `codex exec` result being the fastest of the three shouldn't be over-read as "Coder-Next is raw-faster" — it's one sample, likely favorable output length plus a warm server, not a contradiction of the raw benchmark numbers above.

> **Caveat — don't benchmark back-to-back with other heavy loads:** the first `codex exec` attempt this session measured **4:12** (vs the clean 9.0s above). Cause: two `mlx_lm.benchmark` runs (each memory-mapping ~85-87GB of weights) had just run immediately before it, evicting Coder-Next's pages from the page cache and forcing a slow cold reload under memory contention when the server needed them. Not a model or config problem — an artifact of testing methodology. Let large local-model loads settle before timing the next one.

Only `co`/Codex was switched — `oc`/OpenCode's model is hardcoded in the `home.nix` alias (not read from `config.toml`), so it's still on `Qwen3.6-35B-A3B-8bit` until deliberately changed.

Fallback: both older models (`Qwen3.6-35B-A3B-8bit`, `Qwen3.6-27B-8bit`) are kept on disk, same as always — swap the `model` line in `~/.codex/config.toml` back if needed, no rebuild required (that file isn't Nix-managed).

Codex's fabricated context-window metadata (investigated, not fixed — by design)

Codex's TUI reports "Model metadata... not found" and internally uses `model_context_window: 258400` for any locally-served model — not from omlx, which correctly reports `max_model_len: 65536` via its own `/v1/models` endpoint. Codex's bundled model catalog just doesn't recognize custom/local models and substitutes a generic, provider-agnostic fallback (258400 = 272000 × 0.95), confirmed identical for GPT-5.5

on an unrelated provider in an upstream GitHub issue. Confirmed again 2026-07-12 via `codex debug models`: the entire bundled catalog is only 8 entries, all OpenAI's own hosted models (`gpt-5.5`, `gpt-5.6-sol`, etc.) — no locally-served model can ever match it, so this warning is permanent and provider-agnostic, not specific to whichever local model is currently the default.

Deliberately **not** patched via Codex's `model_context_window` config override: a confirmed upstream bug (openai/codex #16068) means manually setting that value can permanently break auto-compaction after the first context overflow. Safe mitigations instead: prefer `oc` for anything context-heavy (already accurate), and for long `co` sessions run `/compact` manually rather than trusting Codex's automatic trigger. (Almost reintroduced this override during the 2026-07-12 model swap — caught and reverted before it landed; worth re-checking `~/.codex/config.toml` doesn't have a stray `model_context_window` line after any future swap.)

GPU memory & context-window hardening (2026-07-10)

omlx hard-crashed once under memory pressure. Root cause: macOS resets the `iogpu.wired_limit_mb` sysctl to a low default on every reboot, undercutting omlx's own internal ~122GB target, so instead of gracefully rejecting an oversized request it ran out of wired memory and hard-panicked. Fixed with two durable changes:

- `iogpu.wired_limit_mb=124928` (122GB) via a `launchd.daemons` entry in `configuration.nix` with `RunAtLoad = true` — not `system.activationScripts`, which has a known nix-darwin issue where it doesn't reliably re-run after reboot (only on `darwin-rebuild switch`)
- `max_context_window_policy: 65536` in `~/.omlx/settings.json` (under `sampling`) — caps requests server-side so an oversized context is rejected up front instead of exhausting memory

Adding another local model later

```
/opt/homebrew/Cellar/omlx/<version>/libexec/bin/hf download mlx-community/<repo>
omlx restart   # required — server doesn't hot-reload; this triggers a rescan
```

Benchmark before switching defaults:

```
/opt/homebrew/Cellar/omlx/<version>/libexec/bin/mlx_lm.benchmark --model <repo> -p 64 -g 256 -n 3
/opt/homebrew/Cellar/omlx/<version>/libexec/bin/mlx_lm.benchmark --model <repo> -p 11700 -g 256 -n 2
```

---

6. Voice Input — OpenSuperWhisper + Parakeet v3

Local, free, open-source dictation. Installed as a cask, declared in `configuration.nix`.

Model	Why
Whisper V3 Large/Medium/Small	not used — slower, hallucinates during silence
Parakeet v2	marginally more accurate on English only (6.05% vs 6.32% WER), no multilingual
Parakeet v3 — chosen	beats Whisper Large V3 on accuracy (6.32% vs 7.44% WER) at ~23x the speed, zero silence hallucination, adds 25-language support at negligible cost

> **Known limitation:** vocabulary/hotword priming (feeding it jargon like "herdr", "omlx") only works with Whisper models in this app, not Parakeet. Accepted tradeoff — fall back to typing or manual correction on jargon-heavy prompts.

---

7. Not Yet Built — Kokoro TTS (Voice Output)

omlx already bundles `mlx-audio`, which includes the Kokoro TTS model — genuinely on the machine right now, just unused. Neither Claude Code nor Codex has a built-in "read the response aloud" feature. Building it would take a small hook/script: capture an agent's final response text → POST to omlx's TTS endpoint → play the resulting audio. Candidate integration points: a Claude Code Stop hook, or a wrapper skill. Not started, flagged so it isn't forgotten.

8. Why This Is a Different Way of Working

Every point of friction in the loop has a purpose-built fix, and they compose:

- **Typing** → voice input at ~3x typing speed, global and agent-agnostic
- **Reading walls of text** → lavish turns a plan into an annotatable visual artifact
- **Reviewing every diff** → no-mistakes catches what a human reviewer would, with recorded evidence
- **One agent at a time** → treehouse makes parallel worktrees free
- **Managing multiple sessions** → firstmate is a single point of contact for a crew
- **Tool inefficiency** → gh-axi/chrome-devtools-axi cut token cost and latency on GitHub/browser work
- **Cost/availability** → omlx gives Codex and OpenCode a zero-marginal-cost, fully local, fully private backend, with Claude Code still available for the highest-stakes work

None of this replaces judgment — it moves the job from producing and checking individual changes to directing a process that produces and checks itself.

A real day: morning voice-driven planning in lavish → midday parallel scaling across treehouse worktrees → afternoon judgment proportional to risk, not diff size → overnight autonomous gnhf loop → anytime cost-free iteration via `co` / `oc`, reaching for `cc` when stakes call for Claude's ceiling.

---

9. Maintenance, Upgrading & Future-Proofing

> **Golden rule:** everything durable is declared in `~/github/JalenMickey/dotfiles/configuration.nix` (Homebrew taps/brews/casks) or `home.nix` (nixpkgs packages, env vars, session PATH). `homebrew.onActivation.cleanup = "zap"` is intentional and will silently remove anything installed ad-hoc on the next `./rebuild.sh`. Declare before your next rebuild or expect it gone.

> **Never** run `brew install/reinstall/upgrade` at the same time as `./rebuild.sh`. Both touch the Homebrew Cellar and will lock-collide. Wait for one to finish before starting the other.

`sudo` commands (including `./rebuild.sh` itself) fail when run non-interactively — always run these yourself in a real terminal, not via an agent's background shell.

Known gotchas (all hit and resolved across setup sessions)

Symptom	Real cause	Fix
`home.nix` secret-file pattern (`omlx-api-key.local` etc.) silently resolves to `""` after a rebuild, even though the file exists and is readable	`darwin-rebuild switch --flake` runs Nix evaluation in **pure mode** by default; `builtins.pathExists` / `readFile` on any absolute path outside the flake's own store copy silently returns `false`/errors instead of reading the real file — confirmed directly with `nix eval --impure` (true) vs `nix eval` (false) against the same path	`rebuild.sh` now passes `--impure` to `darwin-rebuild switch`. Required for this secret-file pattern to ever work — re-check this flag survives if `rebuild.sh` is ever rewritten
New home.nix env var doesn't show up anywhere, even a fresh terminal *or* fully quitting and reopening WezTerm	herdr is a persistent-session multiplexer (`herdr` launches by "attach to persistent session", not a fresh one) — quitting/reopening the terminal app just reattaches the same long-lived pane process with its original environment. `herdr server stop` doesn't kill that pane either. Even `exec zsh -l` in that pane isn't enough by itself: home-manager's `hm-session-vars.sh` has its own re-source guard (`__HM_SESS_VARS_SOURCED`), and `exec` inherits environment variables from the replaced process — so the guard (already `1` from before the rebuild) blocks re-export of everything, including the fixed value	In the existing pane: `unset __HM_SESS_VARS_SOURCED __HM_ZSH_SESS_VARS_SOURCED; exec zsh -l`. Confirm with `echo $OMLX_API_KEY` (or whatever var) before trusting it

Symptom	Real cause	Fix
App records/detects fine but silently does nothing (e.g. dictation doesn't paste)	macOS Accessibility/TCC grants don't apply to an already-running process	fully quit and relaunch the app after granting permission
omlx broken after a rebuild (bad interpreter path)	undeclared transitive dependency (`python@3.11`) got zap-removed as an "orphan"	declare every real dependency explicitly in `configuration.nix`
global `npm install -g` fails outright	this machine's npm prefix is a read-only Nix store path	use `npx <pkg>` for on-demand invocation
omlx hard-crashed under memory pressure	`iogpu.wired_limit_mb` resets to a low default every reboot	`launchd.daemons` sysctl entry + `max_context_window_policy: 65536` — see §5
`co` breaks with `unexpected argument '--full-auto'`	Codex 0.128 removed `--full-auto`	use `--sandbox workspace-write --ask-for-approval never` — not `--dangerously-bypass-approvals-and-sandbox` , which removes the sandbox entirely
Codex reports fabricated context-window metadata (258400)	bundled catalog doesn't recognize local models, falls back to a generic constant	no safe config fix (upstream #16068); use `oc` for context-heavy work, `/compact` manually in long `co` sessions
`leetcode.nvim` errors "contains listed buffers" on `:Leet`	plugin refuses to launch in a session with other files already open, by design	fixed here via `plugins.non_standalone = true` in `leetcode.lua` — close with `:Leet exit`
`leetcode.nvim` sign-in: pasting the cookie does nothing	Ctrl+Shift+V / Shift+Insert / `"*p` don't reliably paste into the prompt in some terminals	try `<C-r>*` (insert register), or your terminal's native paste; on Brave, strip the leading `cf_clearance=...;` so the string starts at `csrftoken=`
`node` / `npx` abort with `Library not loaded: .../libada.3.dylib` (breaks `npx gnhf` , and `opencode` / `oc` which is a Homebrew node dependent)	Homebrew `node` links `libada.3` , but its `ada-url` dependency auto-upgraded to 4.0.0 (only `libada.4` exists) and node wasn't relinked - Homebrew `onActivation.autoUpdate = true` is the trigger	`brew reinstall node` pulls a current node bottle linked to `libada.4` . Do NOT `brew uninstall node` : `opencode` depends on it. Note this node is undeclared cruft - you declare nixpkgs `nodejs_22` (works at v22, shadowed on PATH), so a rebuild would zap it anyway
`brew reinstall/install` aborts with `Error: unknown install step: remove`	nix-homebrew's patched brew (6.0.1, `zhaofengli/nix-homebrew`) is behind homebrew-core; the current formula uses a post-install step the patched brew doesn't implement	the bottle itself usually still pours and links (node came out usable), so verify with `node -v` before panicking. Durable fix: `nix flake update nix-homebrew && ./rebuild.sh` - never mid-brew

Update commands (verified)

What	Command
Skills (lavish, axi family, skill-creator)	`npx skills update -g`
Codex CLI	`codex update`
OpenCode	`brew upgrade opencode` (declared in configuration.nix, so `./rebuild.sh` also keeps it current)
herdr	`herdr update` (or `herdr channel set stable|preview`)
firstmate	`/updatefirstmate`
gnhf	nothing to do — `npx gnhf` always resolves latest
treehouse / no-mistakes	re-run the original install script (idempotent)
omlx	`brew upgrade jundot/omlx/omlx` — recompiles from source (~3 min); never alongside `./rebuild.sh`

What	Command
Copilot CLI	`copilot update` — the cask is tagged `auto_updates`, so `brew upgrade` won't manage it; the app updates itself
gh, node, everything else in home.packages	`nix flake update` then `./rebuild.sh`

Periodic health checks

- `gh auth status` — tokens can expire
- `copilot login` (re-run if `gc` starts prompting to authenticate — Copilot tokens can also expire)
- `omlx diagnose menubar` — built-in self-diagnostic
- Check disk usage under `~/.omlx/models` and `~/.cache/huggingface` — local models are 20-80GB+ each and accumulate
- Before downloading any new local model: re-run the uncensored/abliterated-branding sanity check from §5
- Re-run `./rebuild.sh` occasionally even with no known changes — catches config drift early

Secrets hygiene

The dotfiles repo is public. Pattern used for `OMLX_API_KEY` — reusable for any future credential:

```
home/<name>.local             # plaintext value, gitignored (home/*.local)
home.nix reads it via:
  lib.optionalString (builtins.pathExists file) (lib.strings.trim (builtins.readFile file))
```

This keeps the *shape* of the config declarative and public while the actual secret value never enters git history.

> **This pattern requires `--impure`.** `builtins.pathExists` / `readFile` on the secret file's absolute path silently fails under Nix's default pure evaluation for flakes — see the gotchas table above. `rebuild.sh` already has this flag; don't drop it when adding the next secret.